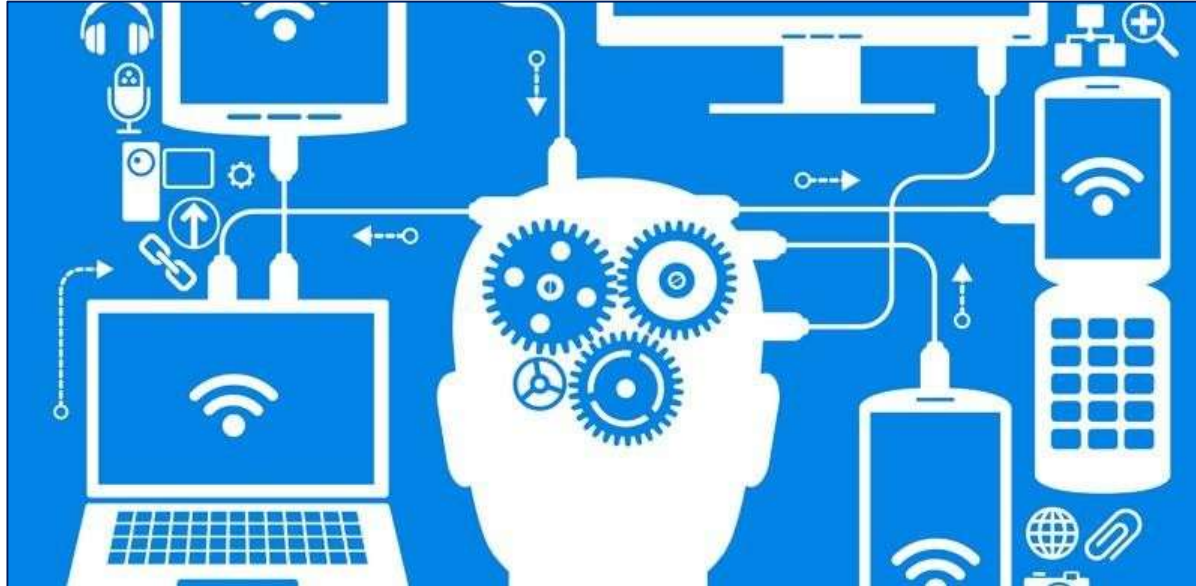


Software Engineering - 5CM505

Lecture 7 – UML # 1

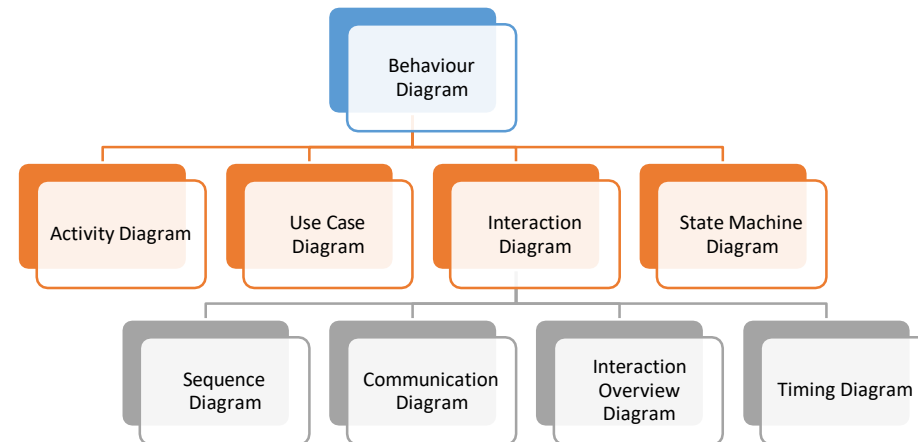
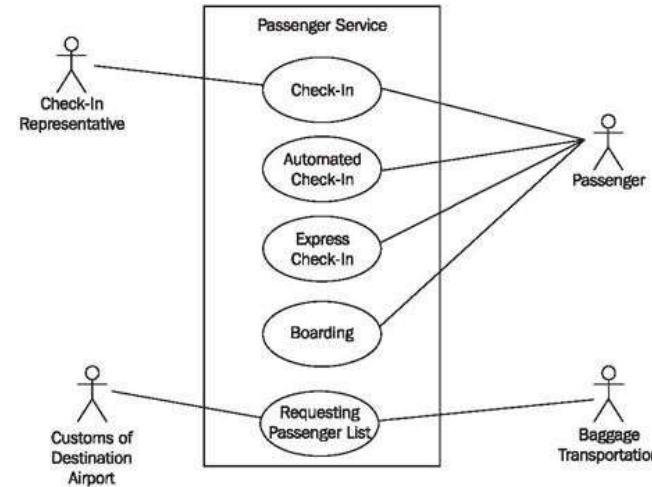
Ioannis Tsioulis (MC)



slides acknowledgements:
Dr Amanda Whitbrook (UoD)
Dr Saeed Ullah (UoD)

Lecture 7 - Content

- Recap from last week
- UML Diagrams
 - Introduction
 - Diagram Categories
 - Hints about using UML diagrams
 - Behaviour Diagrams
 - Use Case Diagram
 - State Machine Diagram
 - Activity Diagram
 - Sequence Diagram
 - Communication Diagram
- Summary

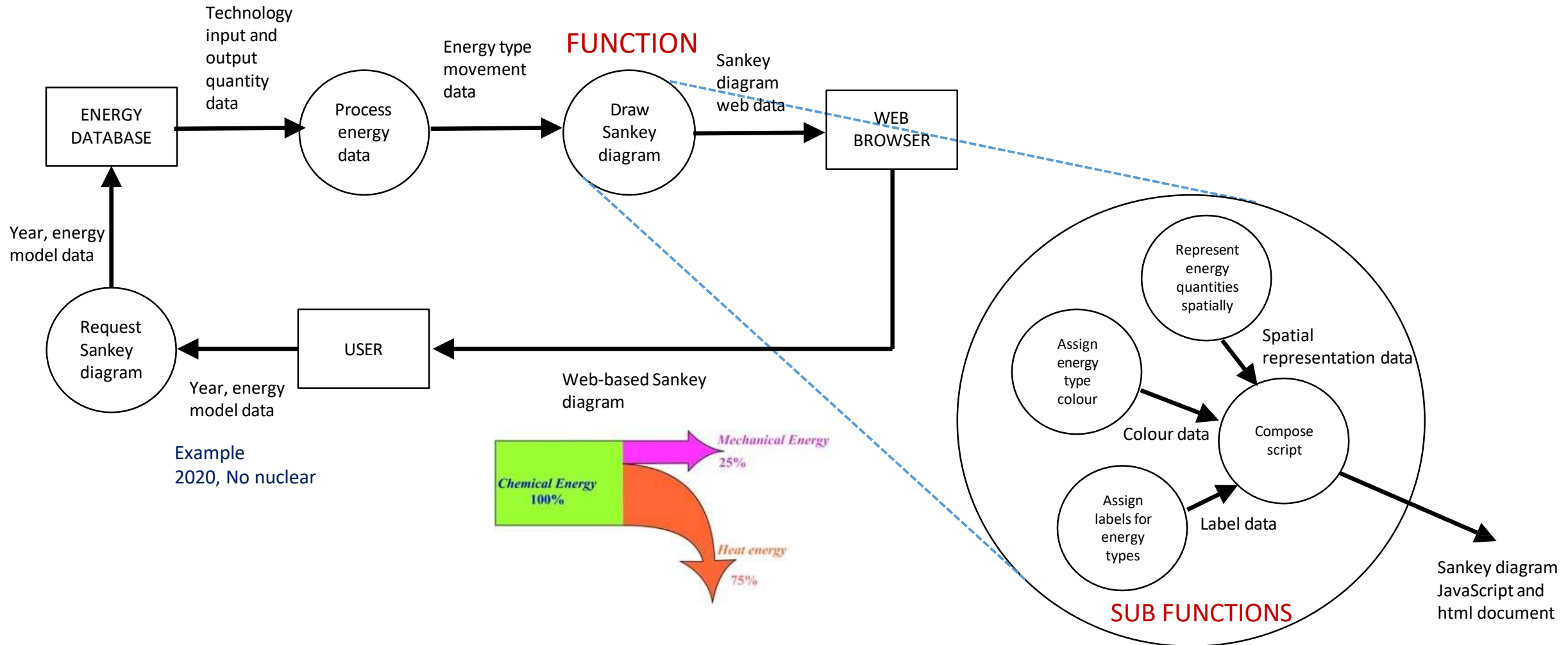


Learning Outcomes for This Lecture

- By the end of this lecture, you should be able to:
 - Translate conceptual systems into the different kinds of UML diagram we have looked at
 - Understand which diagrams are appropriate for modelling the different aspects or views of the system

Recap from Last Week

FFDs



Functional Modelling

ADVANTAGES

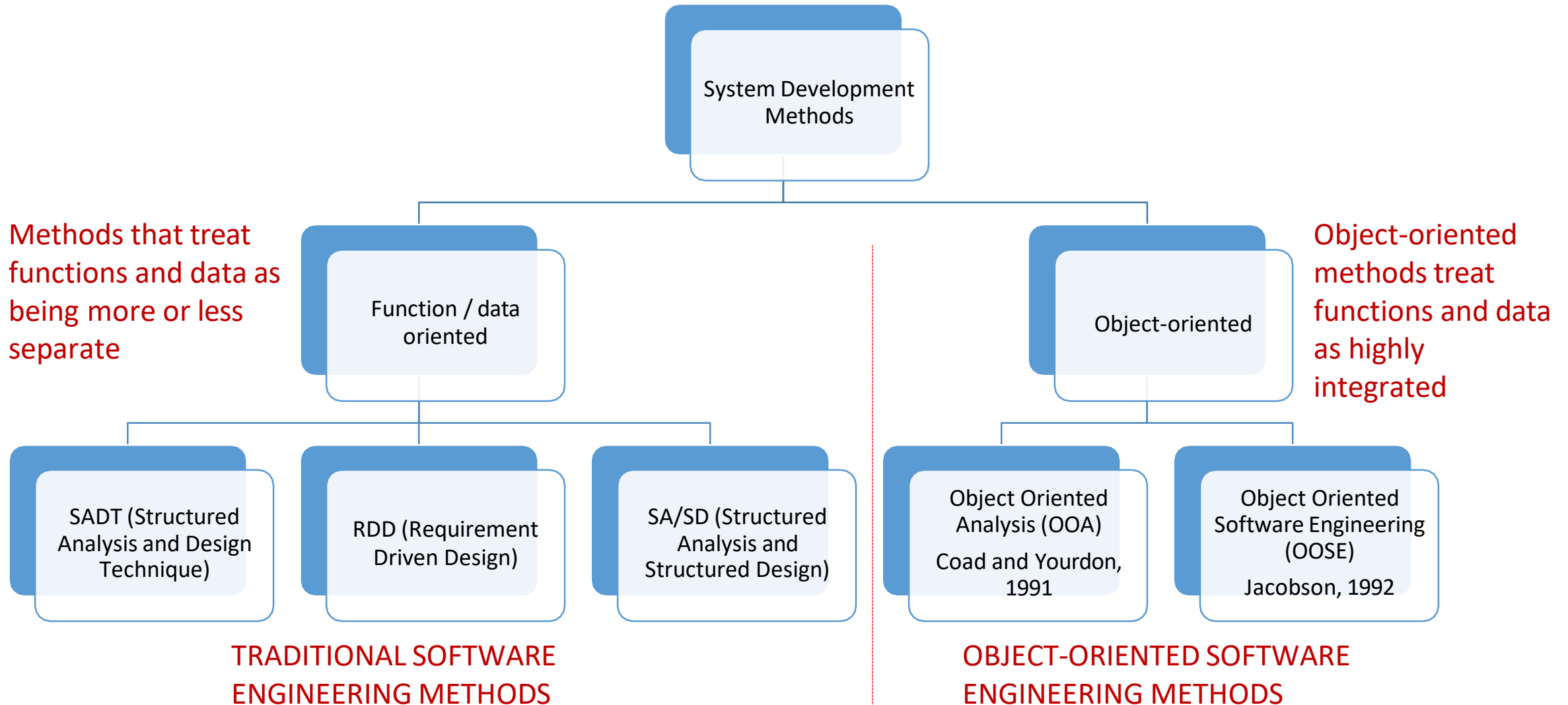
- **Shows relationships between functions**
- Simple technique, which means that the whole team can be involved in the process
- Provides a common understanding of the system
- Customers can also get involved
- Oversights and errors can be detected early
- Identifies the external parts of the system

DISADVANTAGES

- Difficult to verify completeness
- Diagrams can get complicated if not broken down into separate levels properly
- **Functions and data are treated as separate**

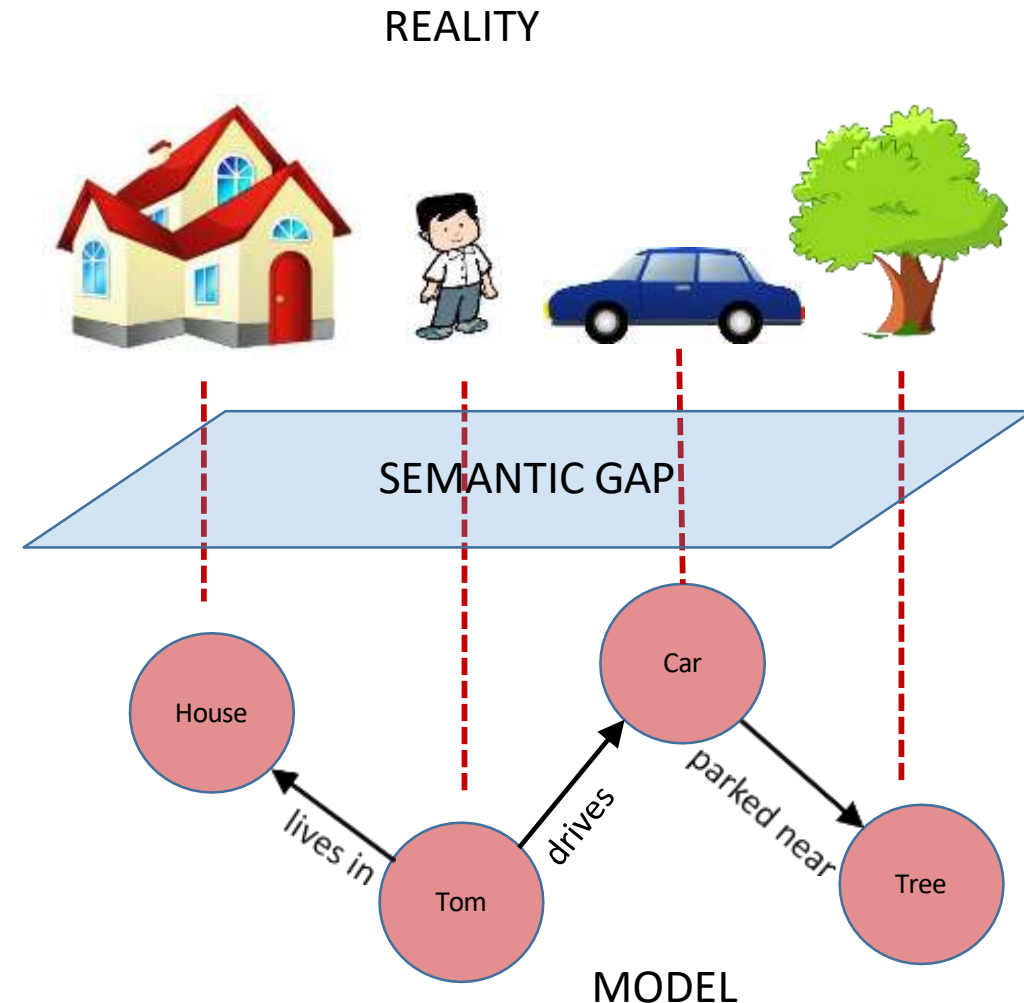


System Development Methods



Object Orientation

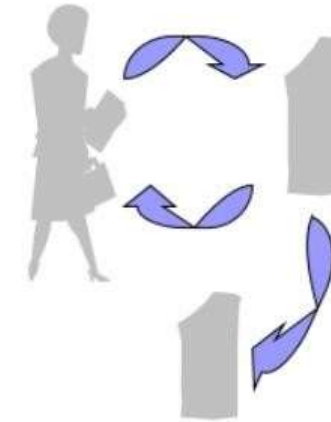
- A technique for **system modelling** where the system has a number of **objects that interact**. It is independent of any programming language
- A model designed using object-orientation is often easy to understand as it can be **directly related to reality**. Often there is only a small gap between reality and the model.



Addressing the Drawbacks of Functional Modelling

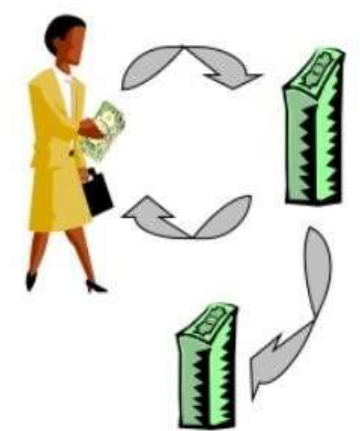
- With object-orientation the requirements are modelled from the **natural language** - what exists in the system is modelled as an **object** and what it does is modelled as a **method** of that object
- The objects that exist in the system are usually stable and change very little
- When changes are needed they usually only affect one or a few objects, which means that, usually, only local changes are necessary

■ Procedural



Withdraw, deposit, transfer

■ Object Oriented



Customer, money, account

UML Diagrams - Introduction

UML Diagrams - Introduction

- Stands for **U**nified **M**odelling **L**anguage
- Adopted as a standard by the Object Management Group (OMG)
- Use of it is widely seen as a core skill for Software Engineers
- The Unified Modelling Language is a visual language for specifying, constructing and documenting the artifacts of system
- Many different types of diagrams but they fall into two main categories:
 - Structure Diagrams e.g. Class Diagram
 - Behaviour Diagrams e.g. Use Case Diagram

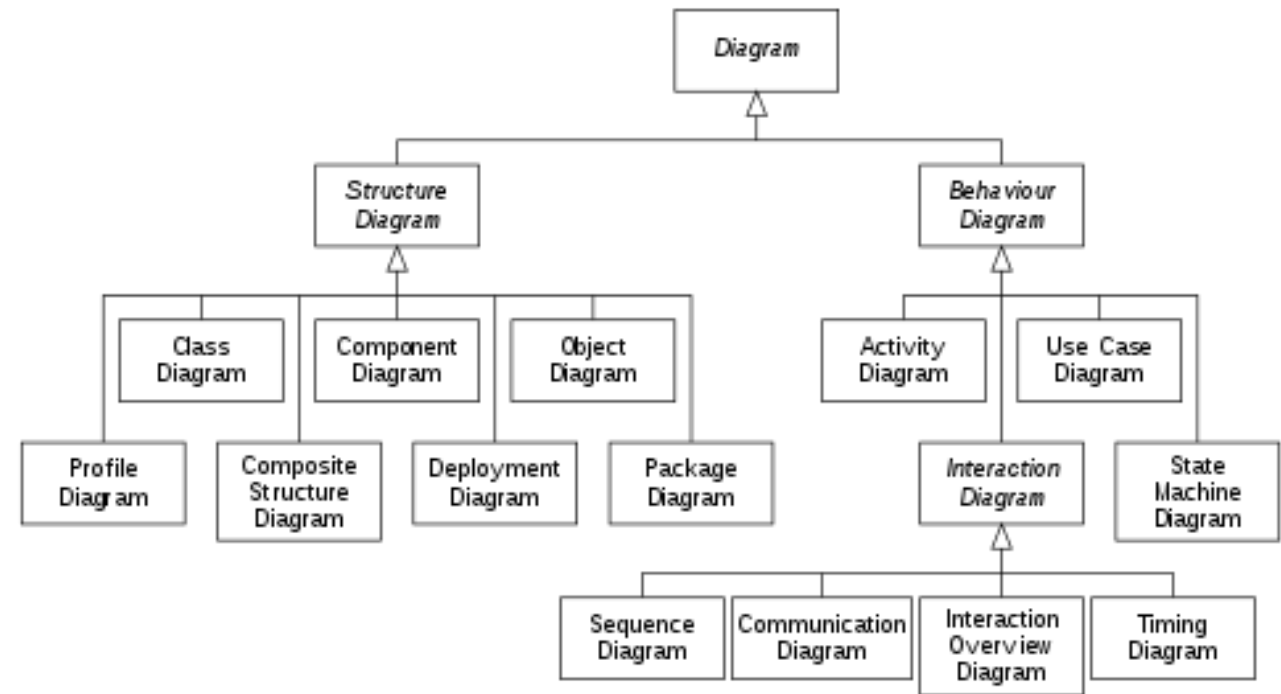
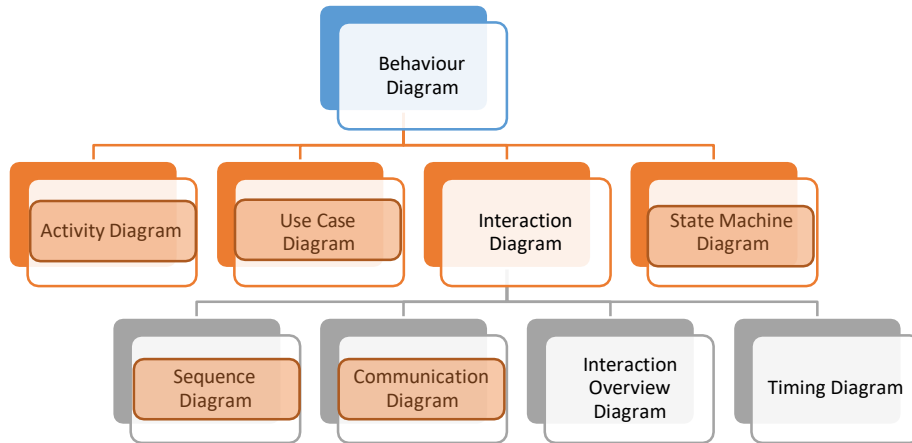
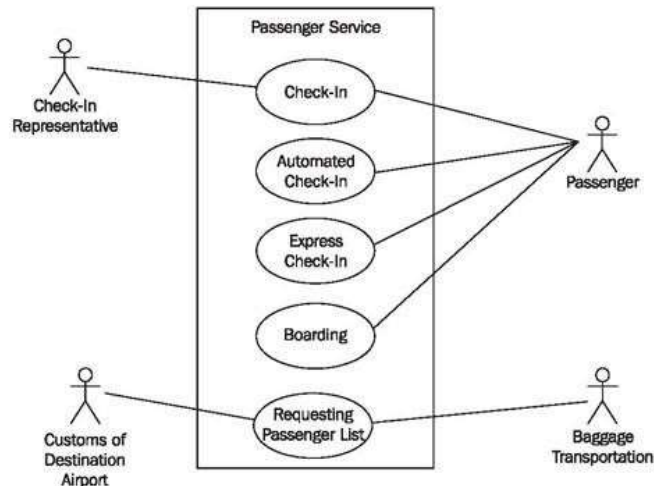


Diagram Categories

Behaviour Diagrams

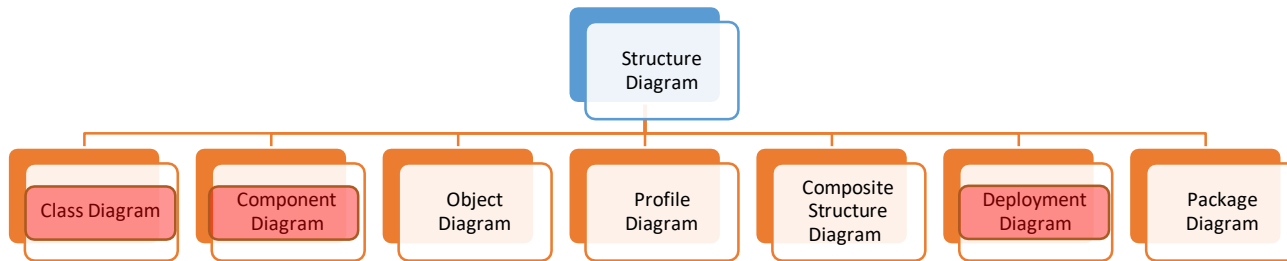


Sample Use Case Diagram

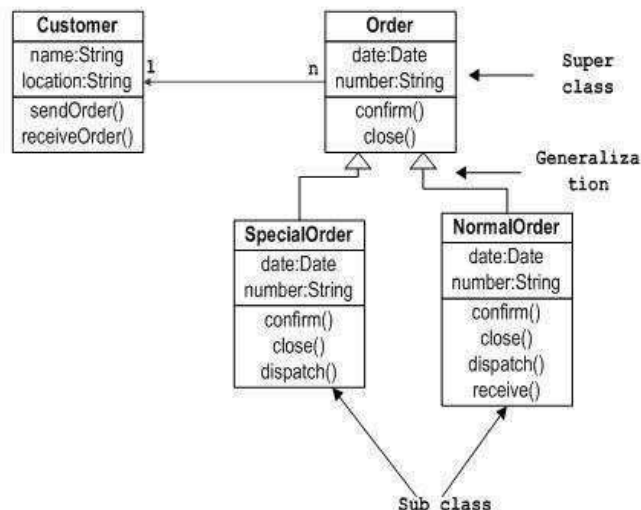


- The focus is on what **must happen** in the system being modelled.
- The behaviour diagrams thus model **functionality or dynamics** of the system
- Interaction diagrams are a subset of behaviour diagrams:
- They emphasize the flow of control and data among the things in the system being modelled.
- For example, the sequence diagram shows how objects communicate with each other in terms of a sequence of messages.
- This week (and next week as well), we will look at some of the diagrams in this category in detail

Structure Diagrams



Sample
Class
Diagram



- The focus is on **what is present** in the system being modelled - what has to be there?
- The structure diagrams are thus concerned with modelling the **system architecture**
- The diagrams view the system from a **static** perspective
- We will cover this category of diagrams next week

Hints for Using UML



1. **UML** is a tool to help produce software. It **provides a common language to communicate about a system** without source code.
2. **However, producing UML is rarely a goal in itself.**
3. In Agile development, UML diagrams should be produced only as needed to communicate effectively. **Their use should expedite development by facilitating group understanding, not slow it down.**
4. There are numerous tools to help generate UML diagrams but the best tools for producing and using UML diagrams are:
 1. Pencils & eraser, plus paper
 2. Whiteboard
5. Although the diagrams have generally understood purposes, and the symbols have understood meanings, **there is no such thing as a “wrong” UML diagram as long as it effectively communicates to the reader.**
6. Feel free to expand, alter, or deviate as needed, as long as understanding is not compromised.
7. If you are arguing about the correctness of the diagram itself, you are wasting time.
8. Argue about what the diagram represents, not about the diagram itself
9. Use text notes on the diagram to explain things further.

Use Case Diagrams

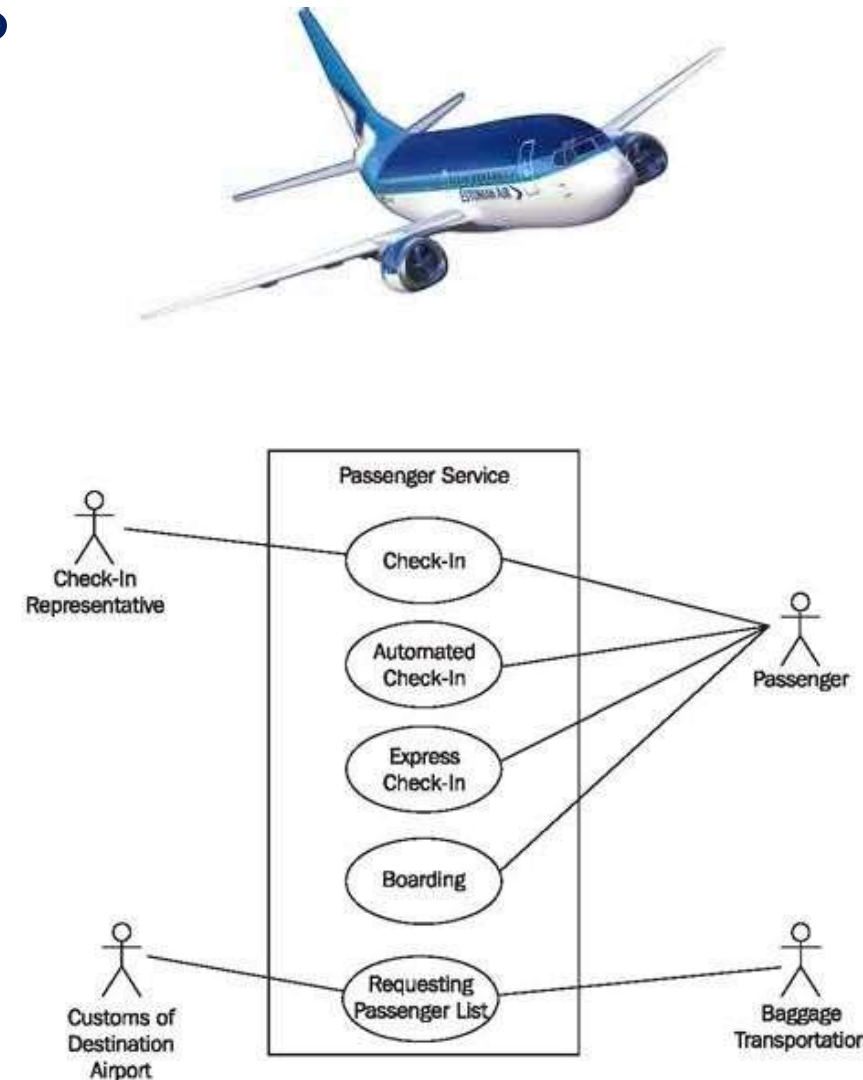
Use Case Diagrams

- Systems must meet the needs of their users
- The **Use Case Diagram** takes a user-oriented approach to systems analysis to help **capture requirements**.
- **Users (actors)** are identified along with the **tasks (use cases)** that they must undertake within the system
- An actor is a user of the system in a particular role. It is a **person** or **another system** (information system, hardware device etc.) that is external to the system being modelled
- The **actor interacts** with the system in some way and thus places demands on it



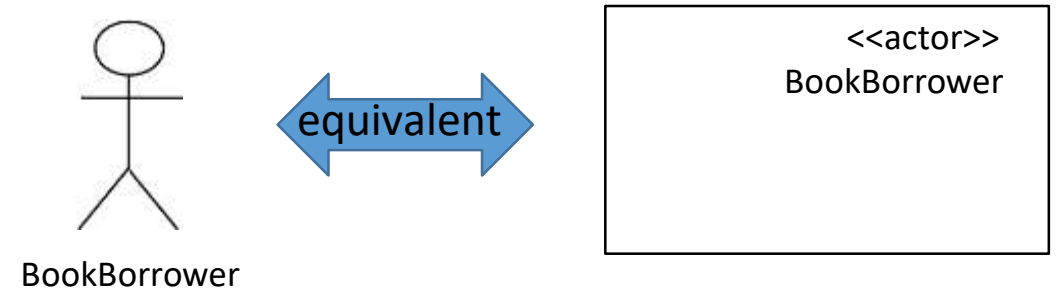
Use Case Diagrams

- A **use case** is a **task** that **actors** need to **perform** with the help of the **system**
- Each **use case** is named after **what happens in a normal, successful instance of the task** being performed
- The detail of each use case should be documented in a third-person active voice
- The information is also recorded pictorially in a **Use Case Diagram**
- **Stick figures** represent **actors**
- **Ovals** represent **use cases**
- There is a **line** between the an **actor** and a **use case** if the actor **takes part** in it
- Can add include, extend and generalisation relationships **between use cases**



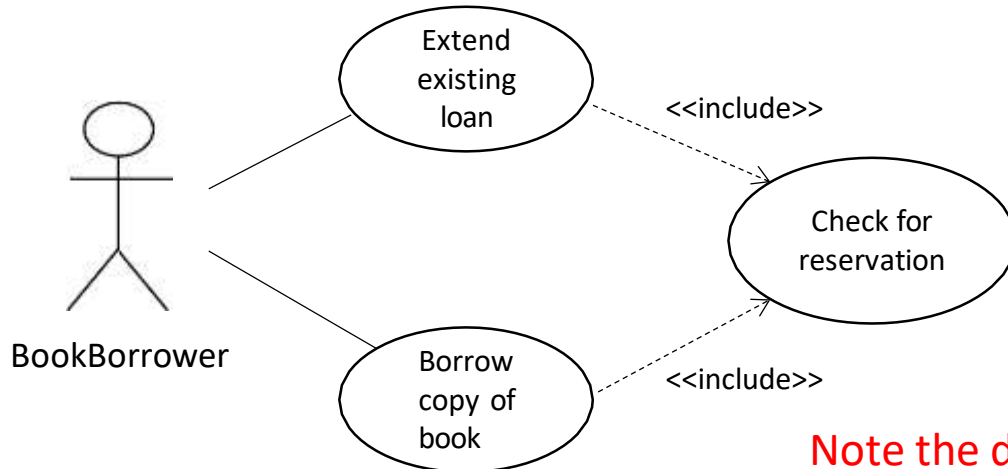
Actors

- **Actors** can also be represented using **square boxes** with the `<<actor>>` heading and the name of the actor
- *These two different notations mean the same thing*



Include Relationships

- `<<include>>` is used when common behaviour is identified from two or more use cases. It is a convenient way to document shared or reused functionality
- Can make use case diagrams smaller and easier to understand



Note the direction of the arrows

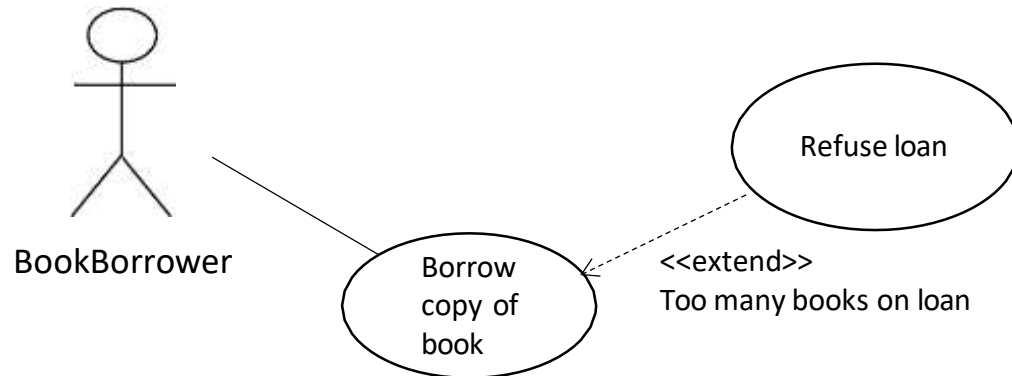
Example: A library system

The actor `BookBorrower` may either extend an existing loan or take out a new book. For reservation use cases there is a need to check whether there is an existing reservation on the book.

If there is a reservation, then the loan on the book cannot be extended. Also, the book cannot be borrowed.

Extend Relationships

- `<<extend>>` is used when a supplementary use case is present that is an extension of the original. It typically defines optional behaviour that is not necessarily meaningful by itself



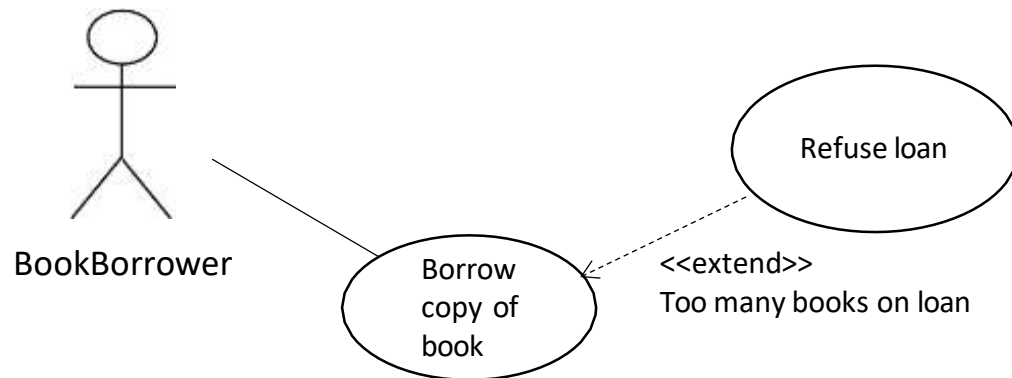
Note the direction of the arrow –
from supplementary case to
normal case

Example: A library system

The actor **BookBorrower** is either allowed to borrow a book or may be refused a loan because he/she has already borrowed the maximum number of books, or maybe there are outstanding fines that have not been paid

Extend Relationships

- `<<extend>>` is used when a supplementary use case is present that is an extension of the original. It typically defines optional behaviour that is not necessarily meaningful by itself



Note the direction of the arrow –
from supplementary case to
normal case

Example: A library system

The actor `BookBorrower` is either allowed to borrow a book or may be refused a loan because he/she has already borrowed the maximum number of books, or maybe there are outstanding fines that have not been paid. UML permits (but does not require) the condition to be shown by the **extension arrow**.

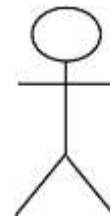
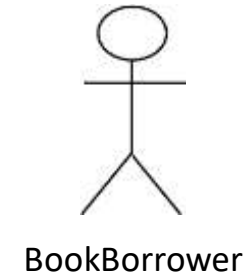
Generalisations

- Two actors or use cases can be related by generalisation (a more specific actor / use case and a more general one). These relationships are shown by a **solid line with a closed arrow head pointing to the more general case**

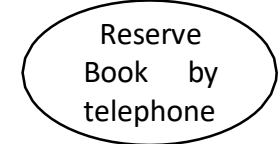
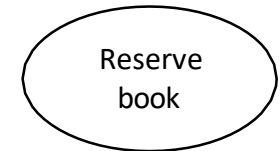
Example: A library system

The actors `JournalBorrower` and `BookBorrower` are related as people entitled to borrow journals are also entitled to borrow books.

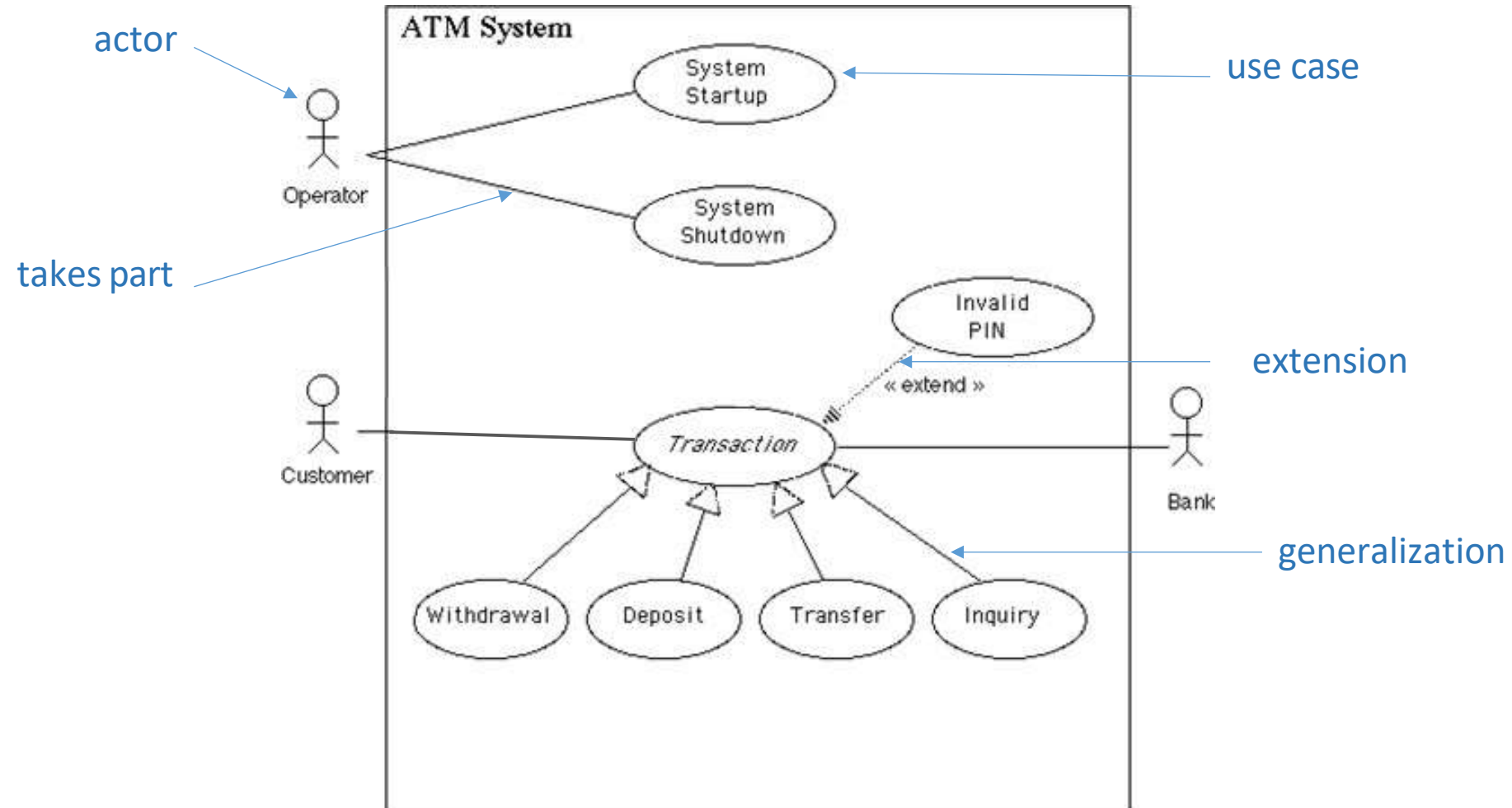
The use cases `Reserve book` and `Reserve book by telephone` are related.



JournalBorrower



Example Use Case Diagrams



Group Exercise

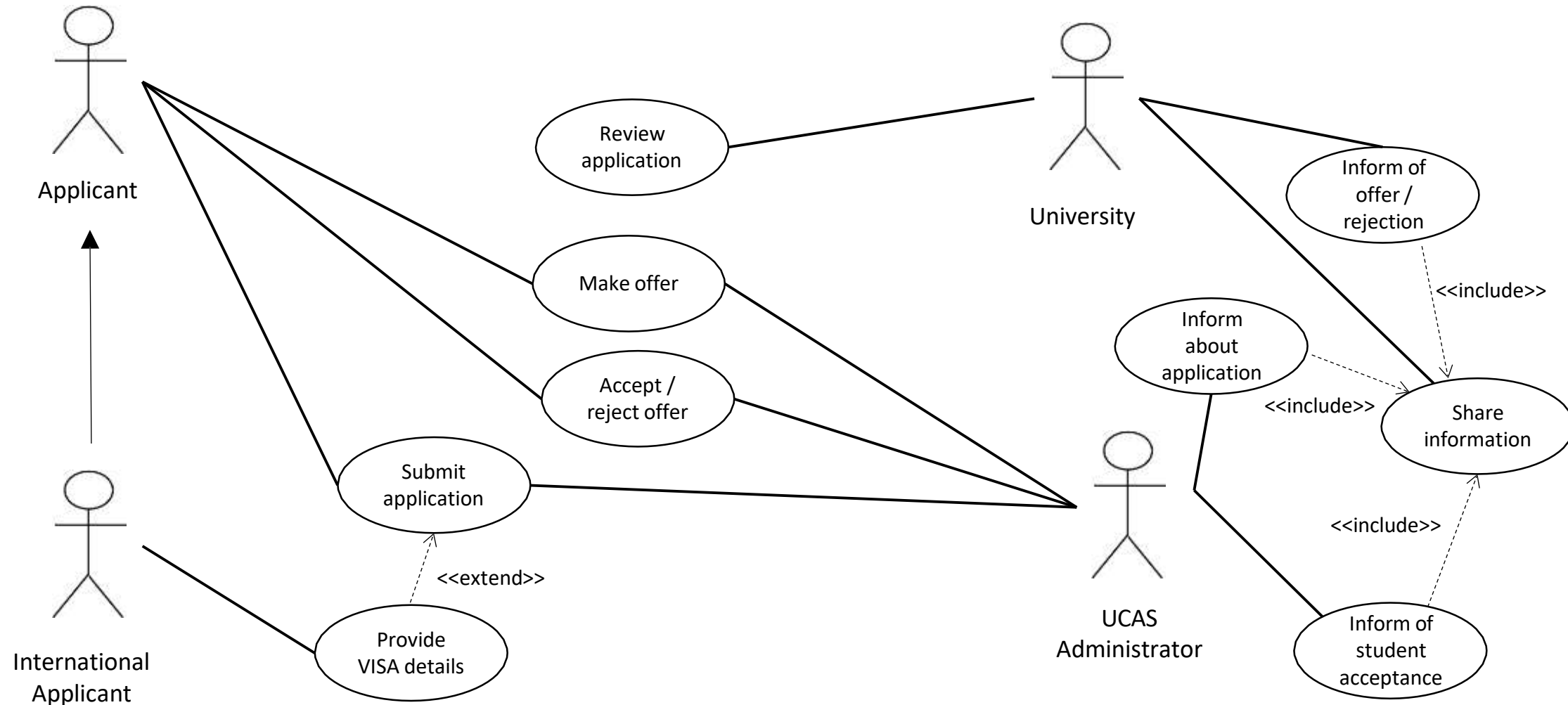
- Draw a simple use case diagram either:
 - For a UCAS application system
 - For the system you performed object oriented analysis on last week

(10 minutes)

The **Universities and Colleges Admissions Service (UCAS)** is a UK-based organisation whose main role is to operate the application process for British universities.

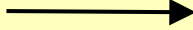
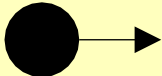
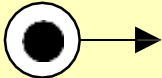


UCAS Use Case Diagram



State Machine Diagrams

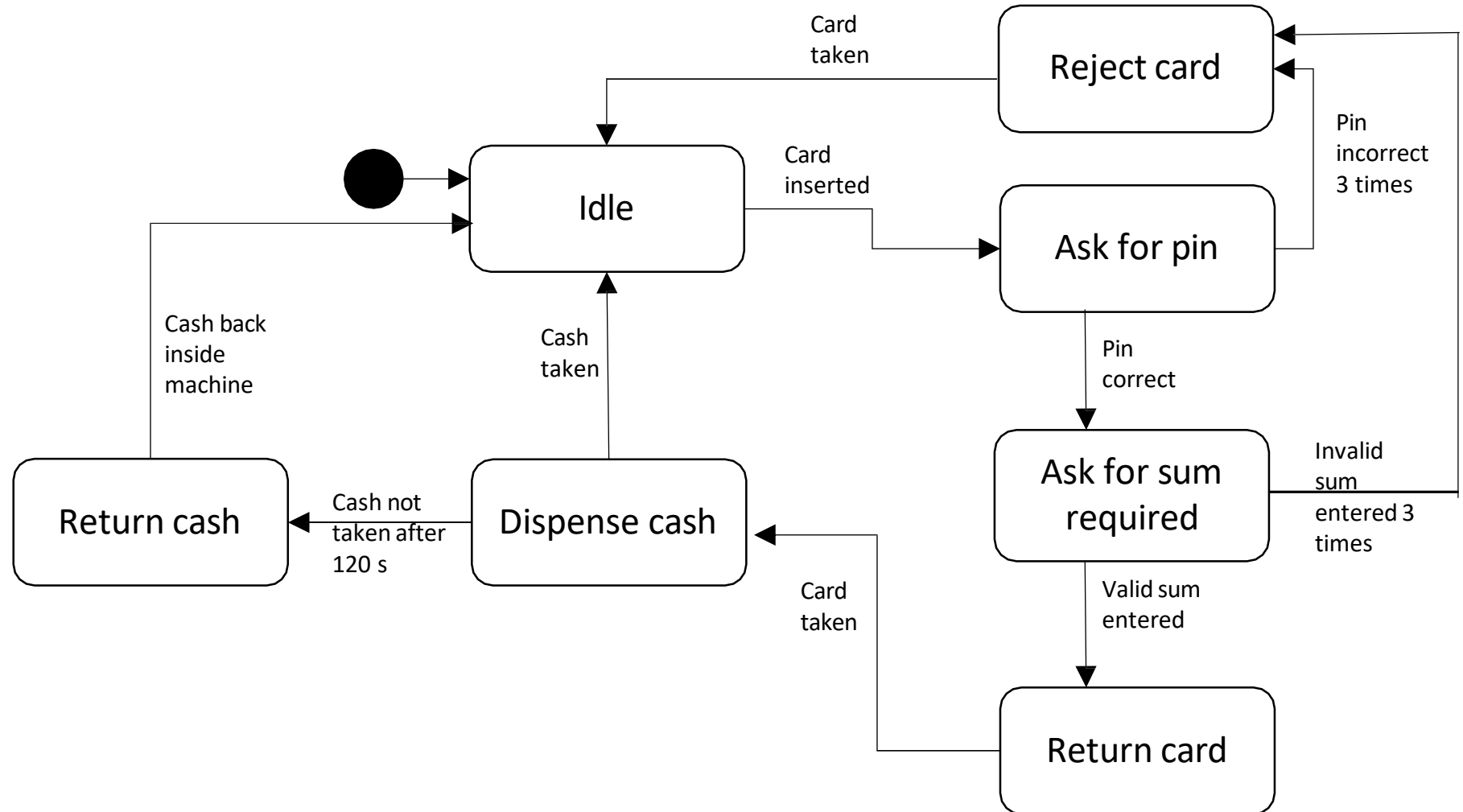
State Machine Diagrams

- **State Machine Diagrams** model the **state** of an object and how the **state changes** (*transitions to another*) in **response to different events** that occur within a system
- In State Machine UML Diagrams:
 - **States** are shown as **boxes with rounded corners** 
 - **Transitions** between states are shown as **arrows** 
 - **Events** are shown by writing their **names** alongside the transition arrow
Event
 - A **Start Marker** (a **black circle with an unlabelled arrow** pointing towards a state) shows the **initial state** of the object. 
 - Optional **Stop Marker(s)** can also be used to show the **final state(s)** of the system. These are represented by a **black circle with a ring** around it. 

State Machine Diagram Example



ATM Machine



Group Exercise

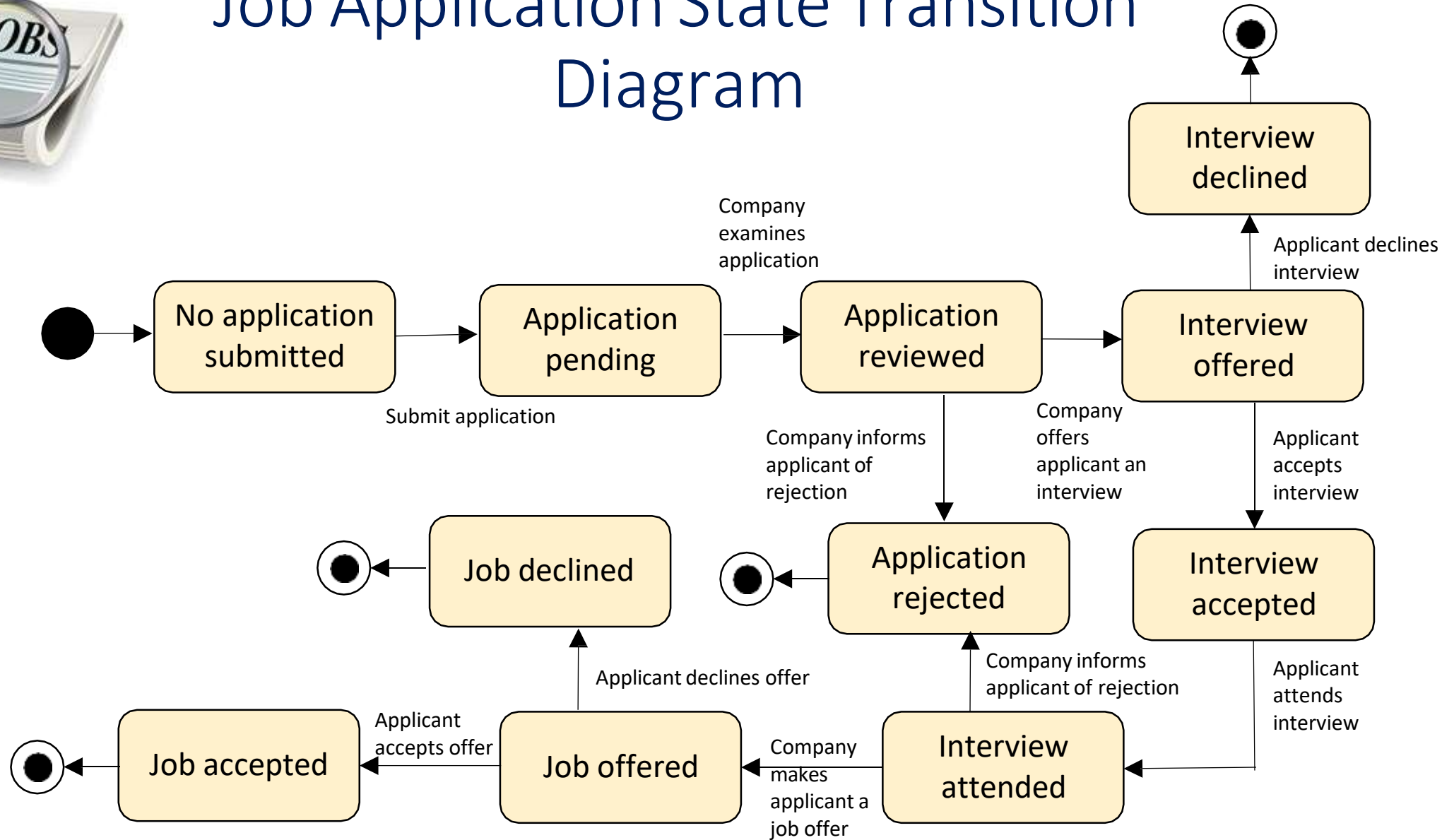
- Draw a state machine diagram either:
 - For the process of applying for a job

(10 minutes)





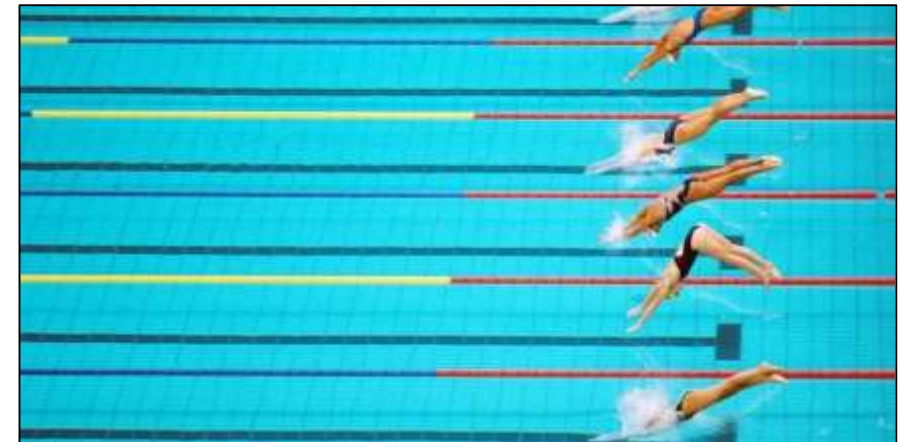
Job Application State Transition Diagram



Activity Diagrams

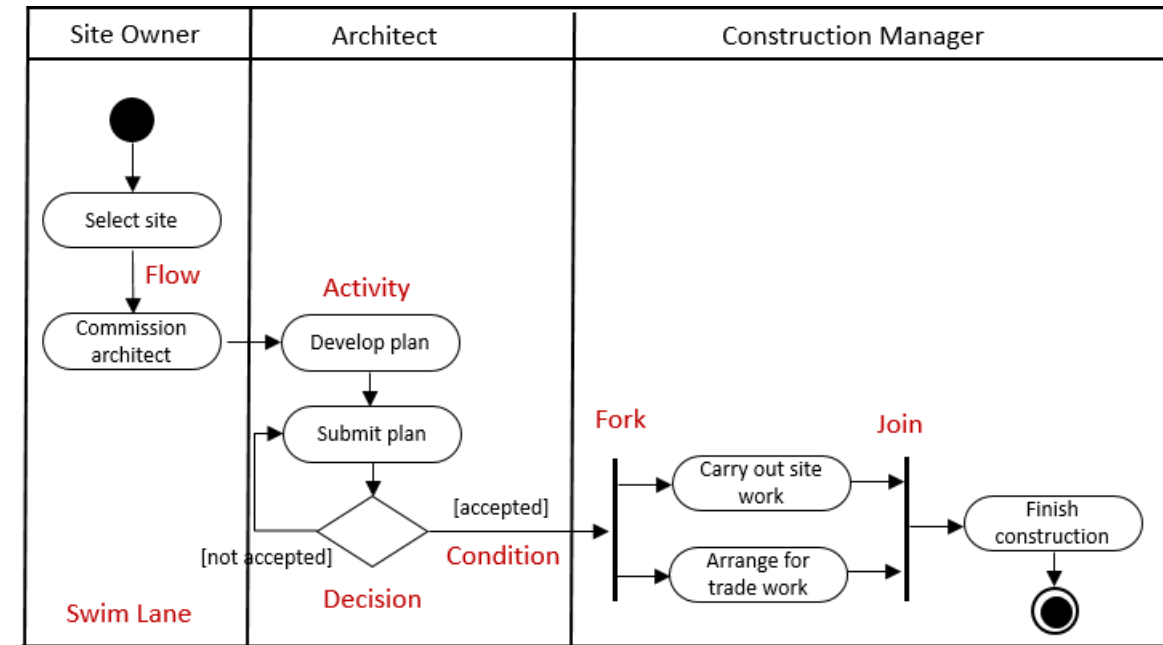
Activity Diagrams

- **Activity diagrams** represent the **flow of control** from one activity to another
- They are often used for **modelling the logic** in a particular **use case** or **usage scenario**
- The diagram can be partitioned into ***swimlanes*** based on the **actors completing the activities**, although these are optional.



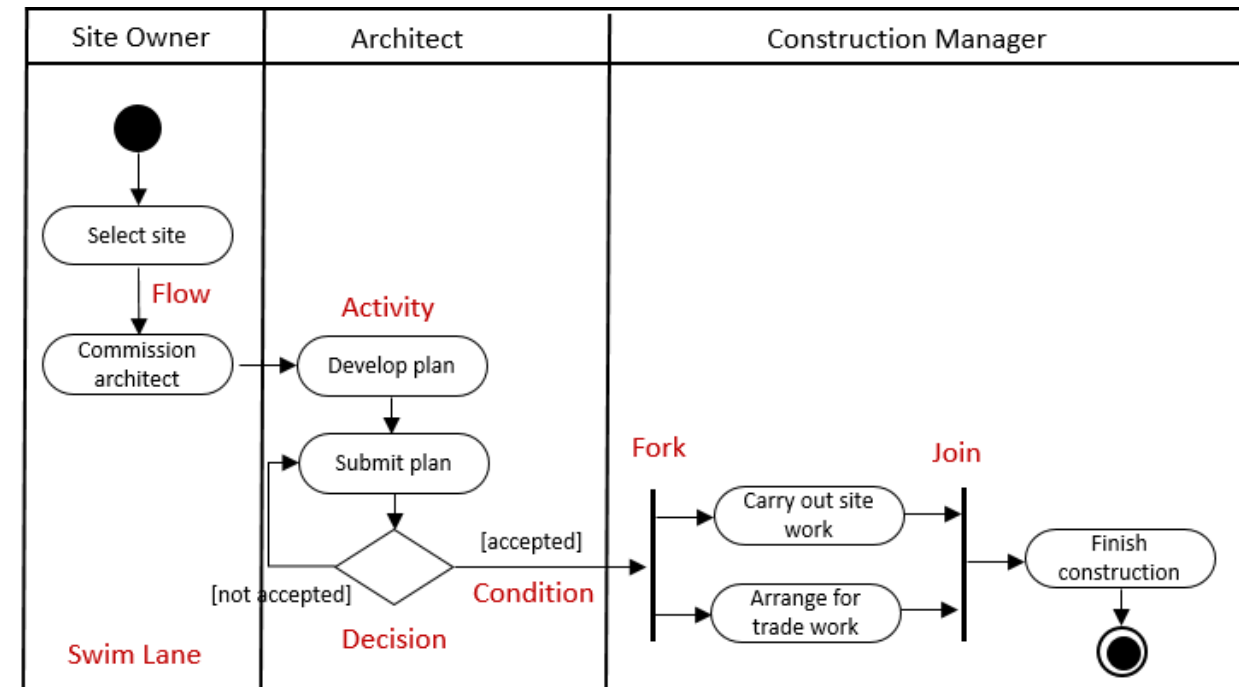
- **Initial node.** The arrow from the filled in circle points to the starting node in the diagram.
- **Final node.** A filled circle with a border represents an ending point. There can be zero or more final nodes in an activity diagram.
- **Activity.** Rounded rectangles represent activities. An activity may be physical, such as “Inspect Forms”, or electronic, such as “Display *Create Student Screen*”.
- **Flow.** These are represented by arrows on the diagram.

Components #1



- **Fork.** A black bar with one flow going into it and several leaving it. This denotes the beginning of parallel activity.
- **Join.** A black bar with several flows entering it and one leaving it. All flows going into the join must reach it before processing may continue. This denotes the end of parallel activities.
- **Decision.** A diamond with one flow entering and several leaving. The flows leaving include conditions
- **Condition.** Text such as [Incorrect Form] on a flow, defining a guard that must evaluate to true in order to traverse the node.

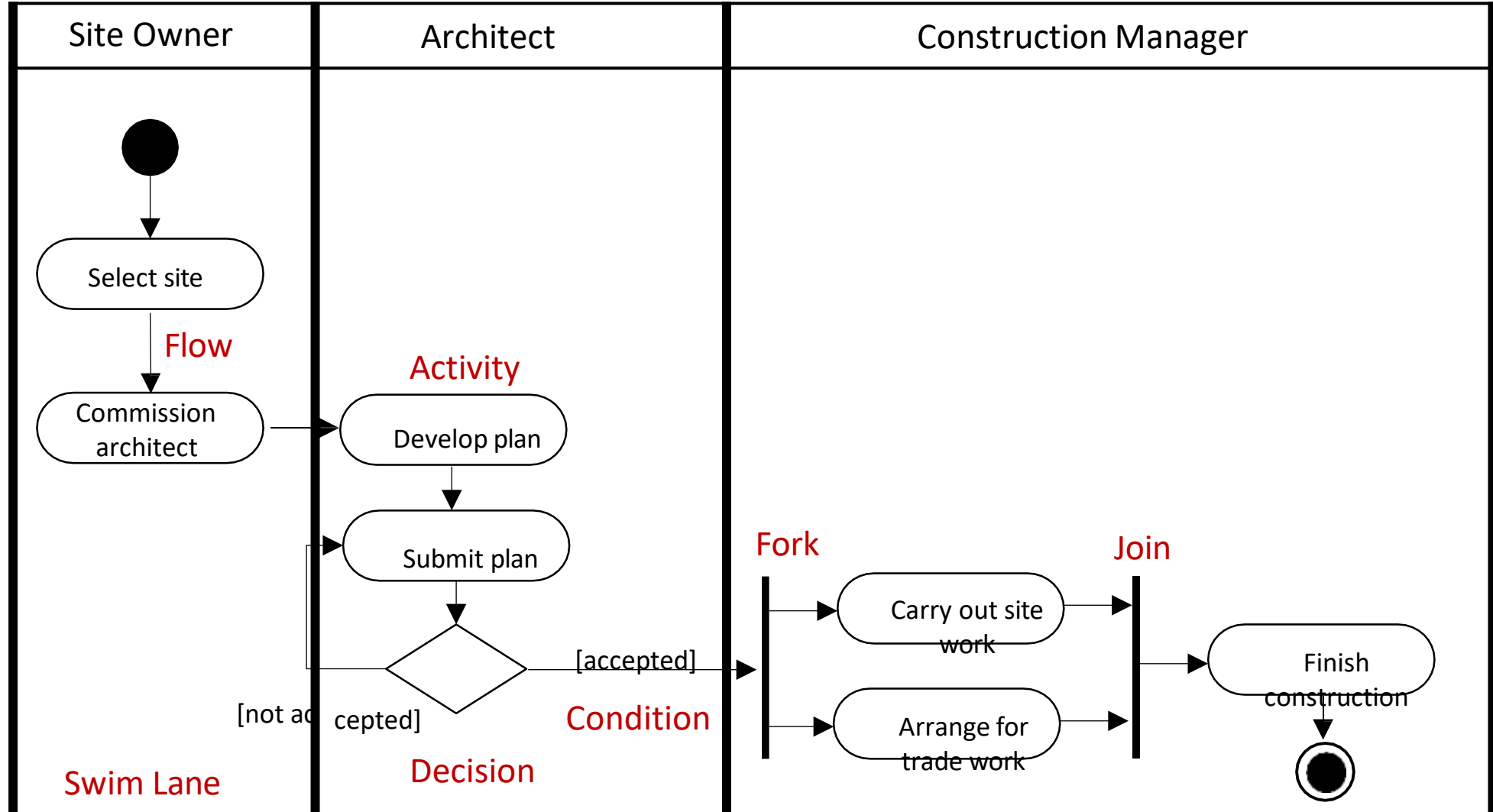
Components #2



Activity Diagram Example #1



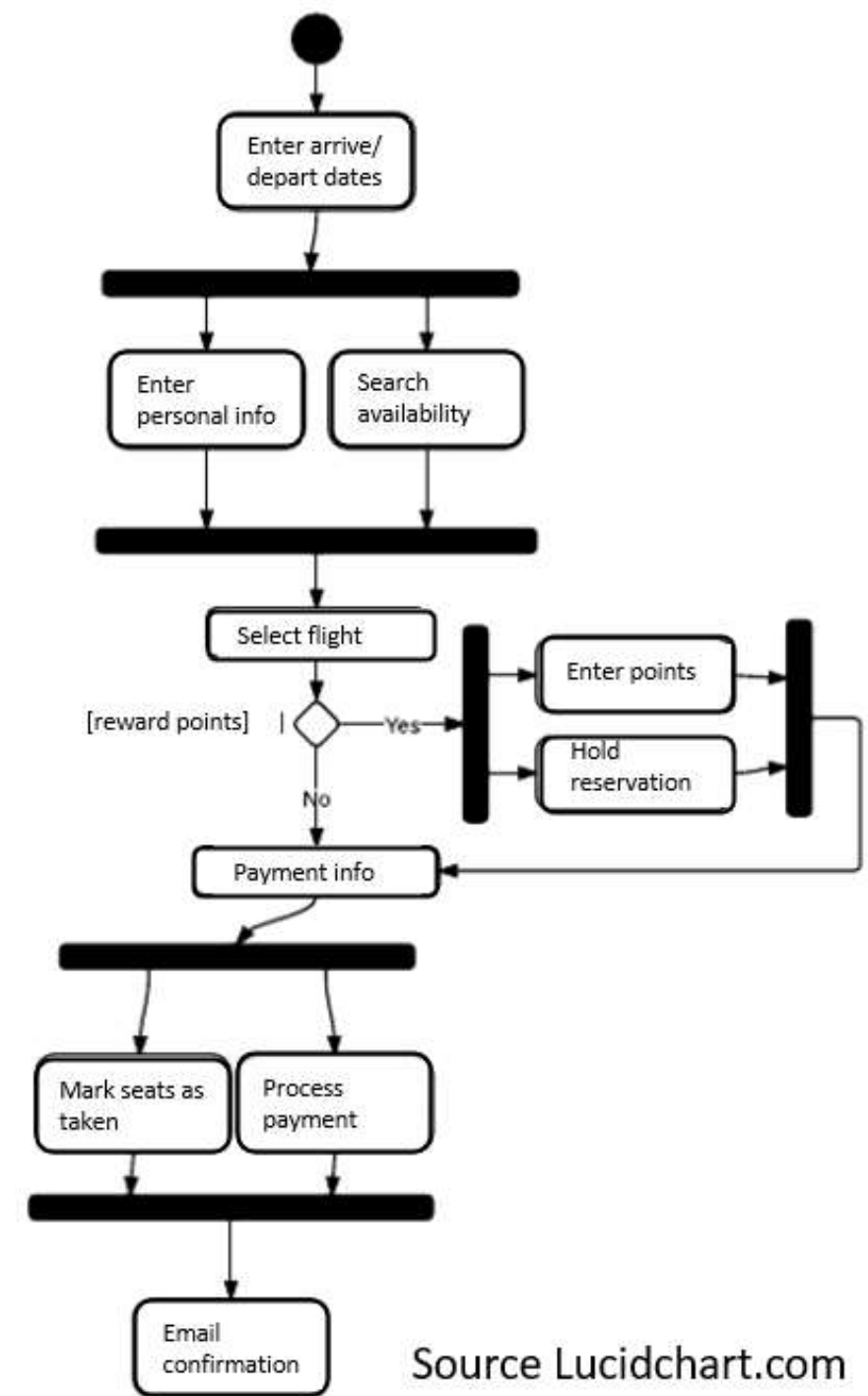
Construction



Activity Diagram Example #2



Online flight reservation



Group Exercise

- Draw an activity diagram either:
 - For the process of placing an online order, where the actors are the customer, the company and the company's shipping department

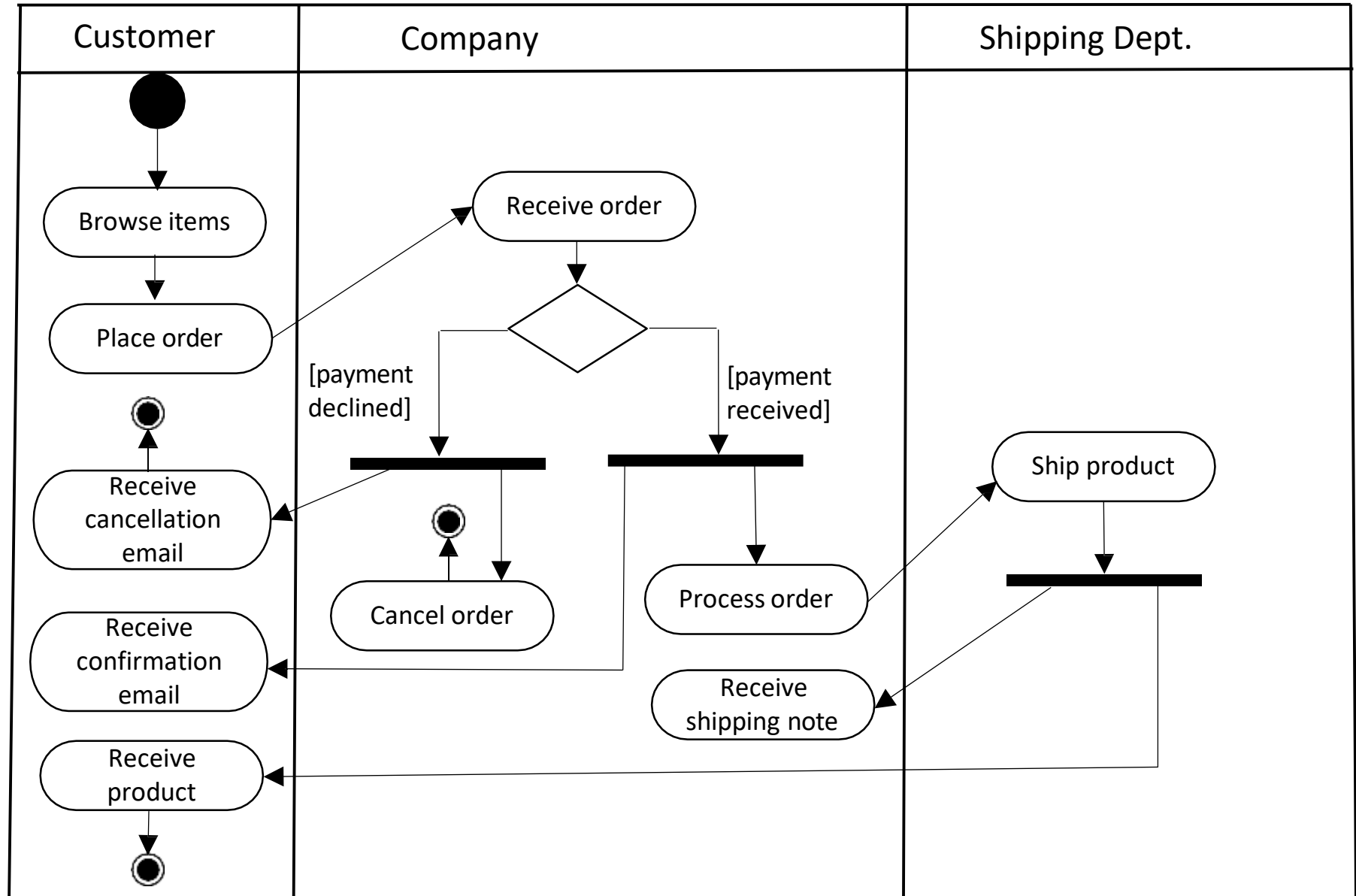
(10 minutes)



Activity Diagram Example

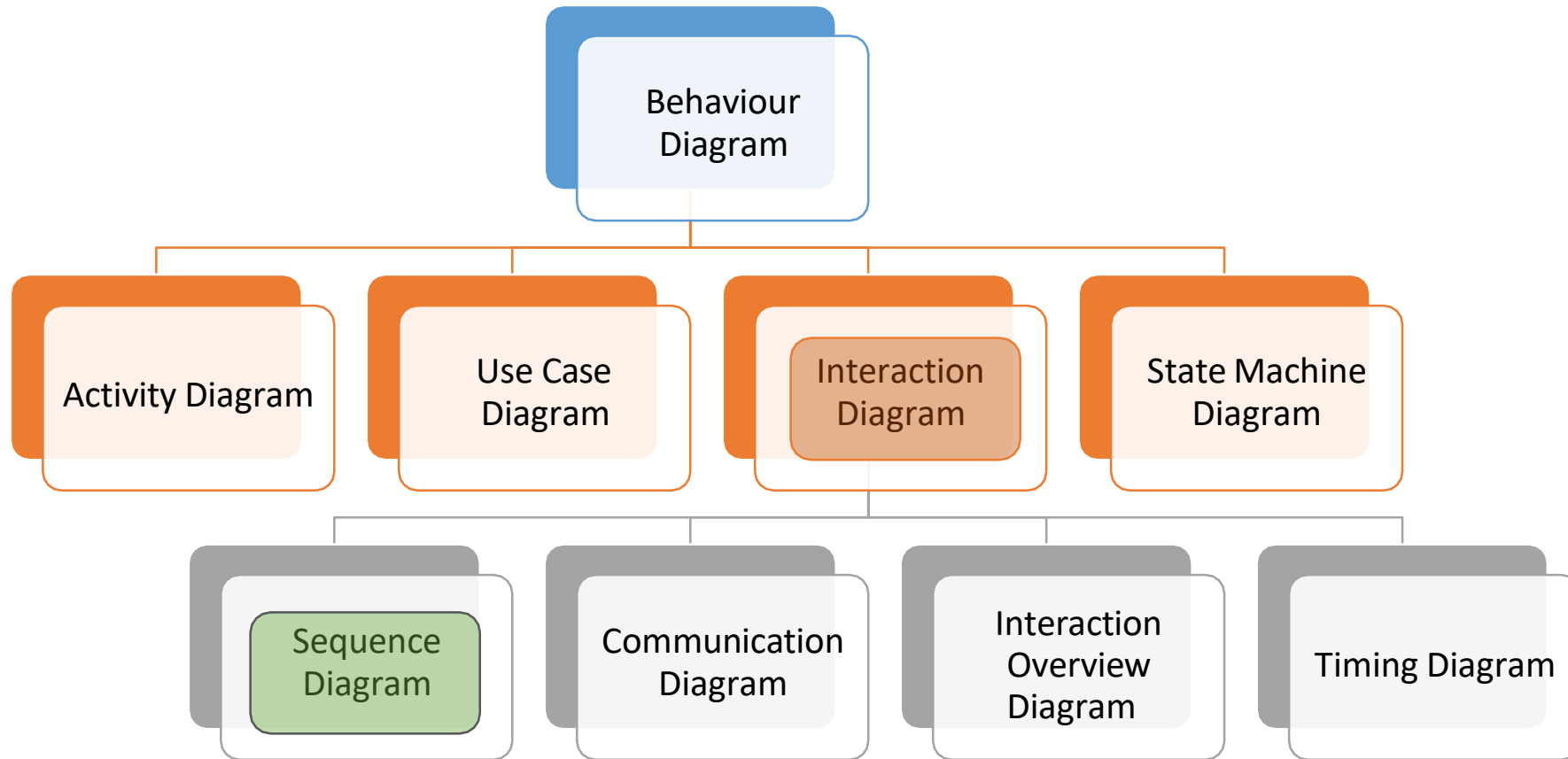


Placing an order



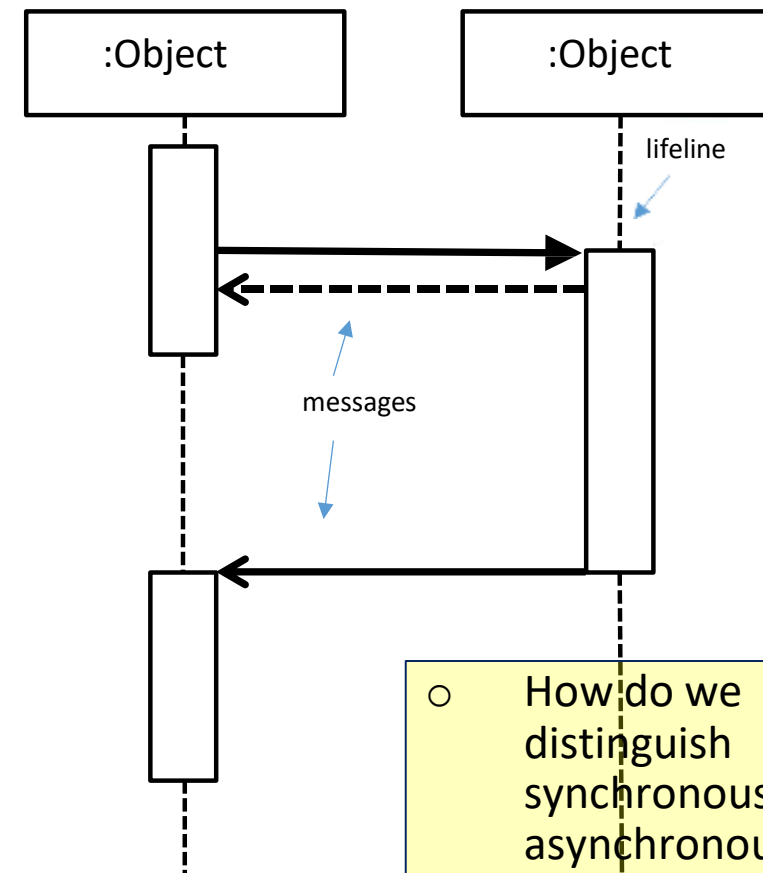
Sequence Diagrams

Behaviour Diagrams



Sequence Diagrams

- **Sequence diagrams** are sometimes called *event diagrams* or *event scenarios*.
- They are a type of **interaction diagram**
- They show how **objects** in a system **interact** with one another and in what **order** those interactions take place, i.e. the interactions are arranged in a time sequence. Time passes from the top of the diagram top the bottom
- They also show the **sequence of messages** exchanged between the objects in the system in the order they occur (horizontal arrows)
- The different **processes** or **objects** that live **simultaneously**, are represented as parallel vertical lines or **lifelines**
- The diagram allows the specification of simple runtime scenarios in a graphical manner.

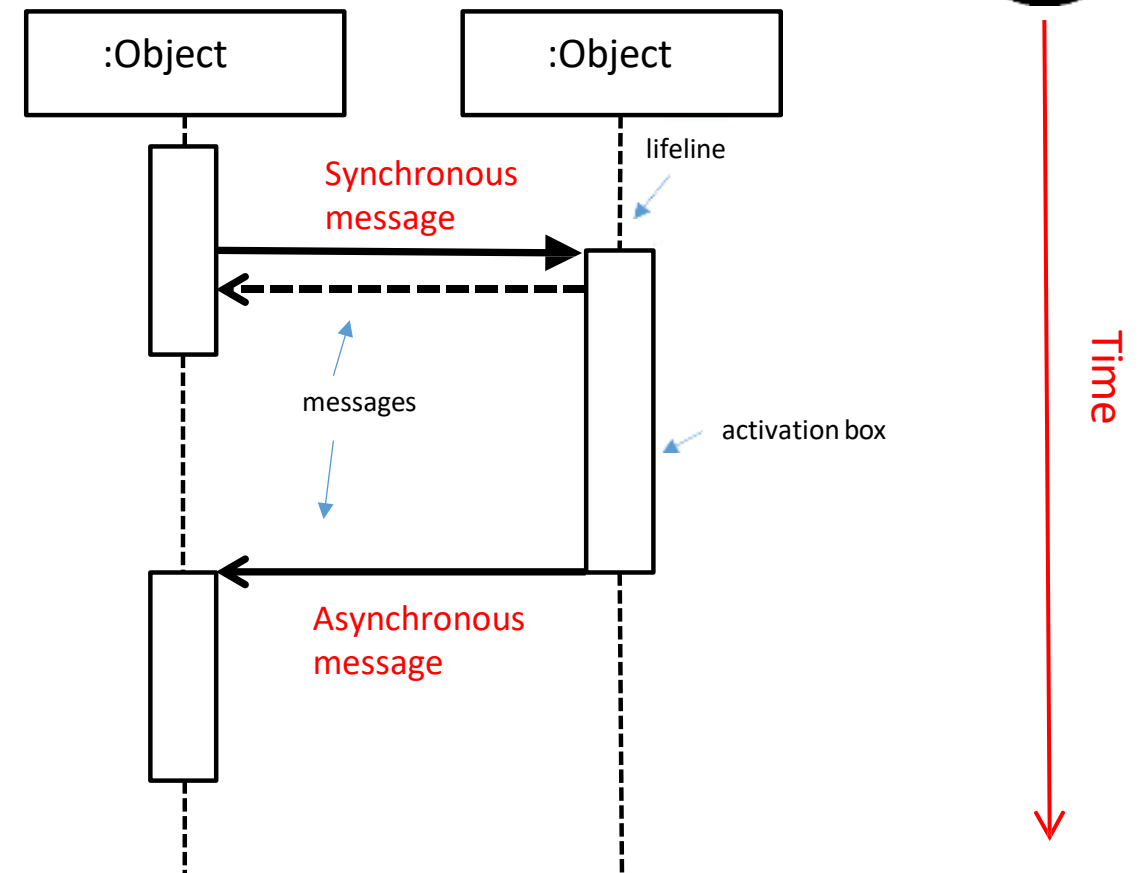


- How do we distinguish synchronous and asynchronous messages?



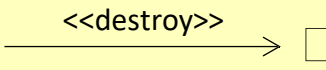
Sequence Diagrams - Messages

- **Messages** are written with horizontal arrows with the message name written above them
- Dashed lines are sometimes used to represent reply messages.
- If a caller sends a **synchronous message**, it must wait until the receiver has processed the message to continue. Solid arrow heads represent synchronous calls
- If a caller sends an **asynchronous message**, it can continue without having to wait for the receiver to process it. Open arrow heads represent asynchronous messages
- **Activation boxes**, or **method-call boxes**, are rectangles drawn on top of lifelines to represent that processes are being performed in response to the message



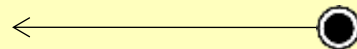
Sequence Diagrams – Other Message Types

- Delete Message

- A message that destroys an object. 

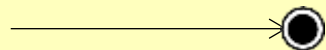
- Found Message

- A message sent from an unknown recipient.



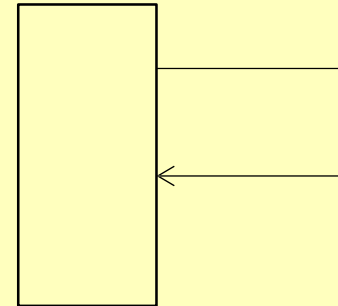
- Lost Message

- A message sent to an unknown recipient.



- Self Message

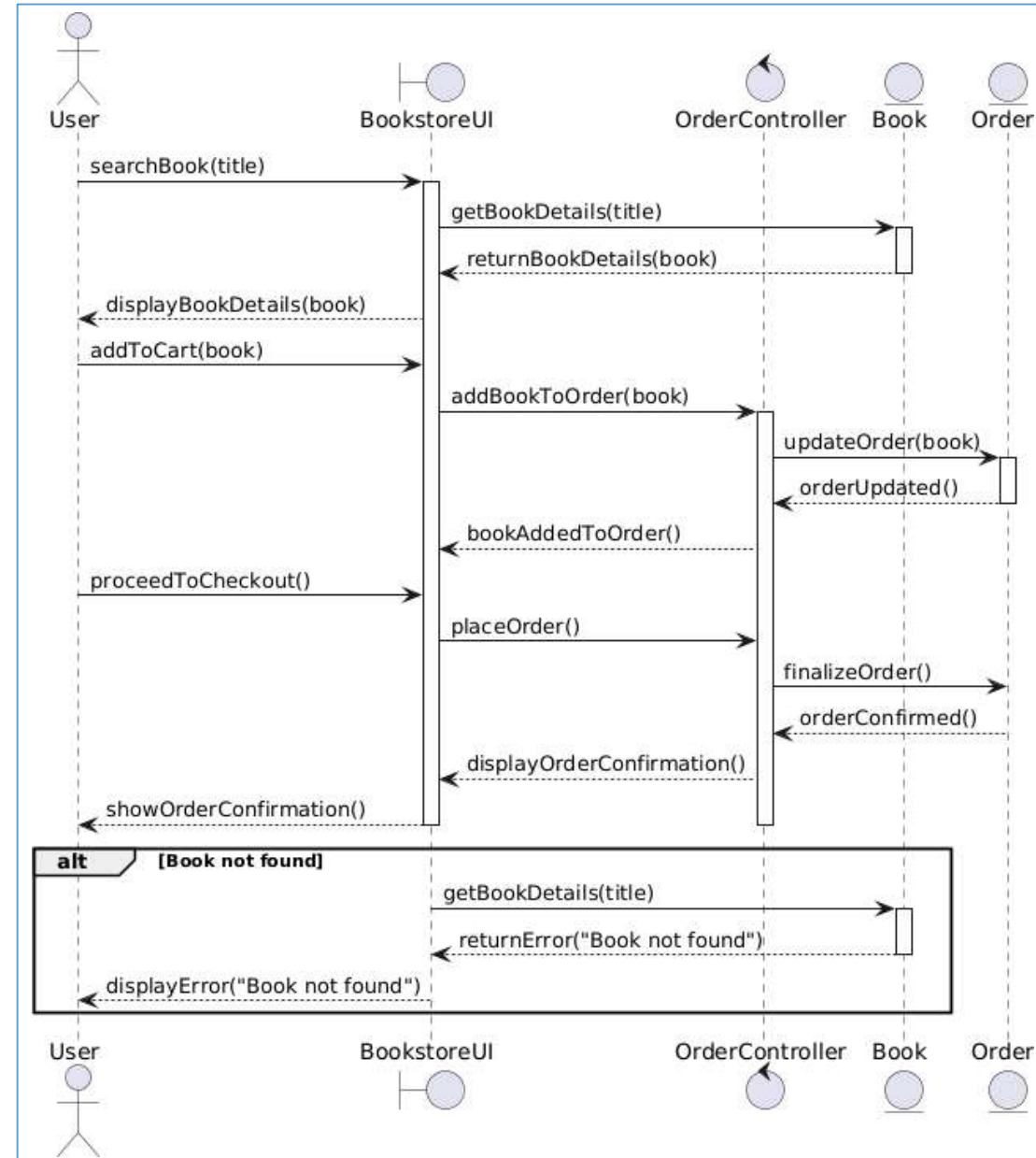
- A message an object sends to itself.



Sequence Diagram Example #1



Online Bookstore



Group Exercise

- Create a sequence diagram for the process of withdrawing cash from a cash machine

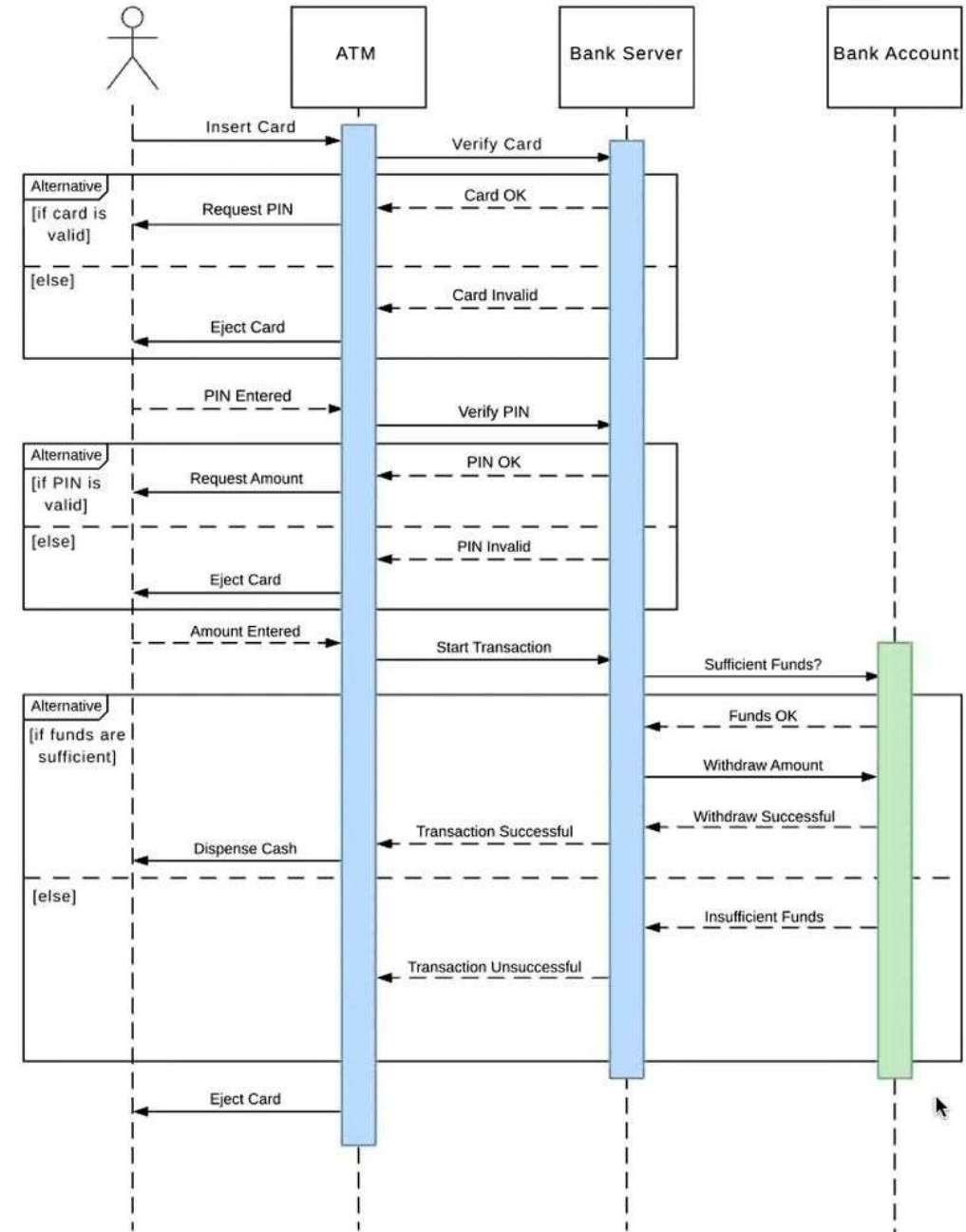
(10 minutes)



Sequence Diagram Example #2

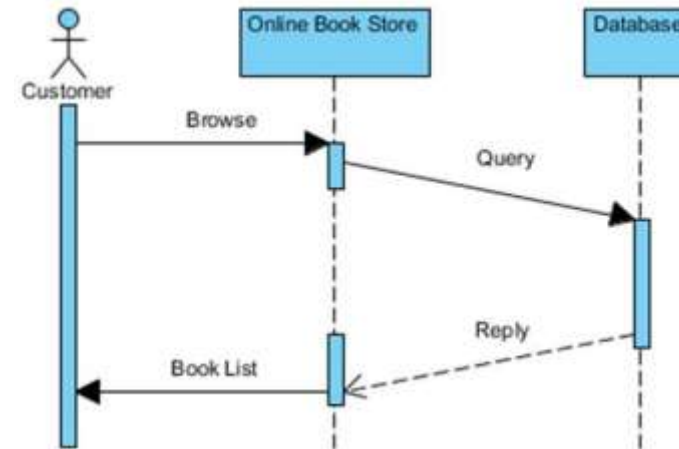


ATM Machine

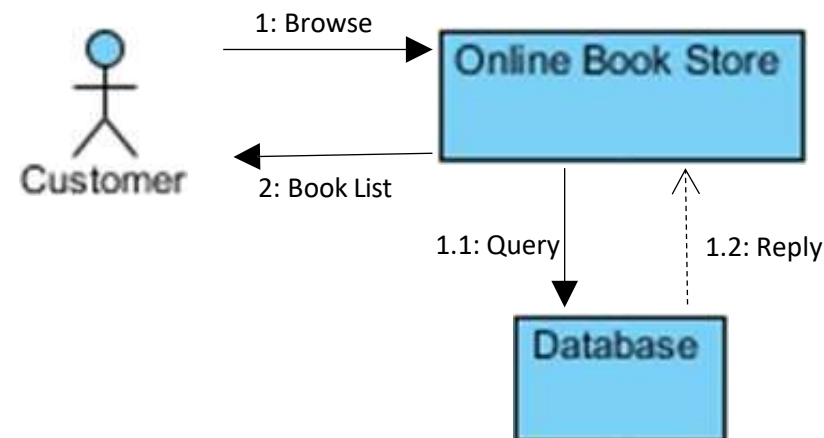


Communication Diagrams

- As in the Sequence Diagram, a **Communication Diagram** models the **interactions** between objects in terms of **sequenced messages**.
- However, the ordering of messages in these diagrams is maintained by labelling messages with a **chronological number** placed near the link the message is sent over.
- Reading a communication diagram involves starting at message 1.0 and following the messages from object to object.
- Communication diagrams show a lot of the same information as Sequence Diagrams, but because of how the information is presented, some of it is easier to find in one diagram than the other.



Sequence
Diagram



Communication
Diagram

Group Exercise

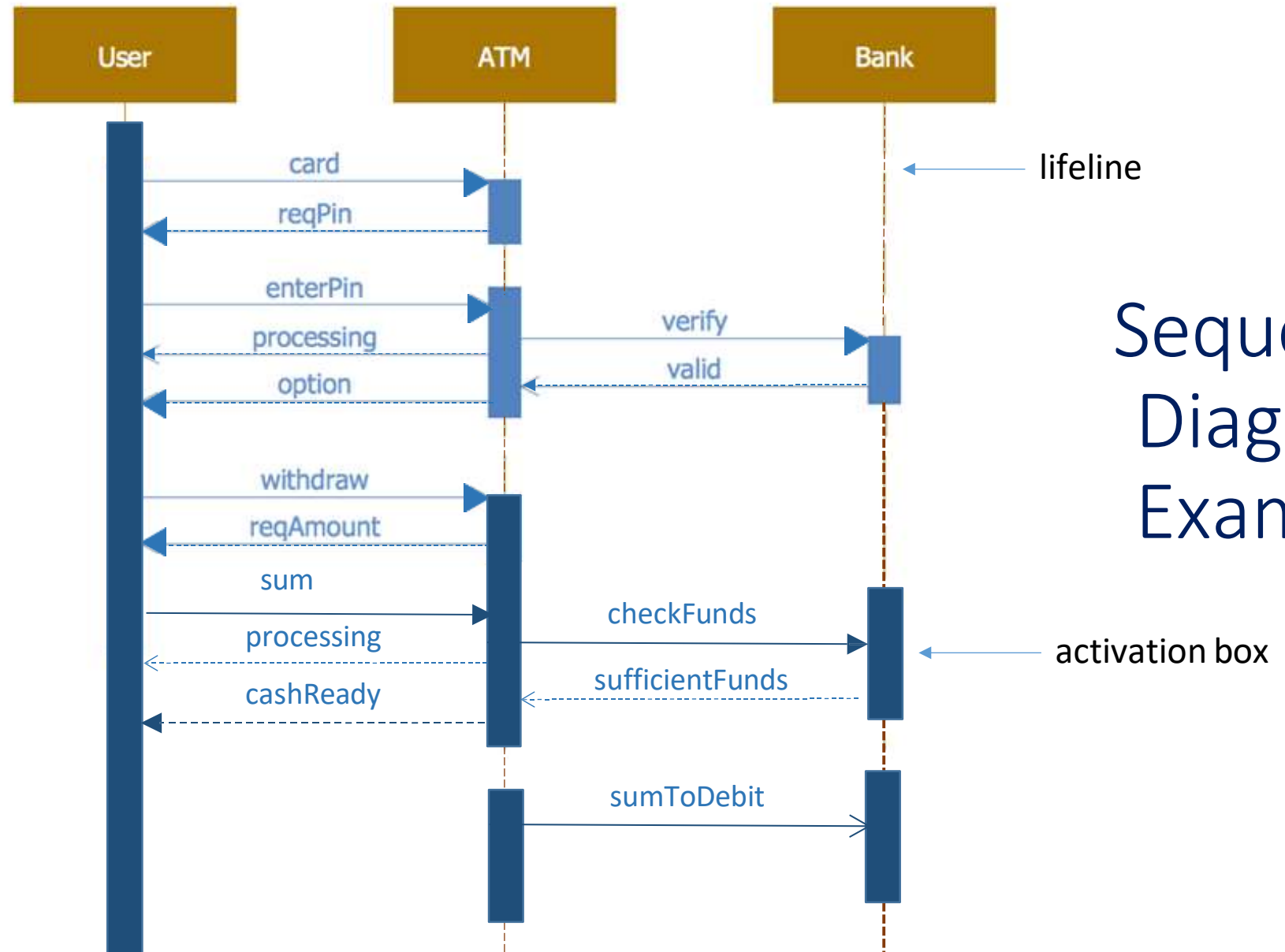
- Turn the ATM sequence diagram into a communication diagram

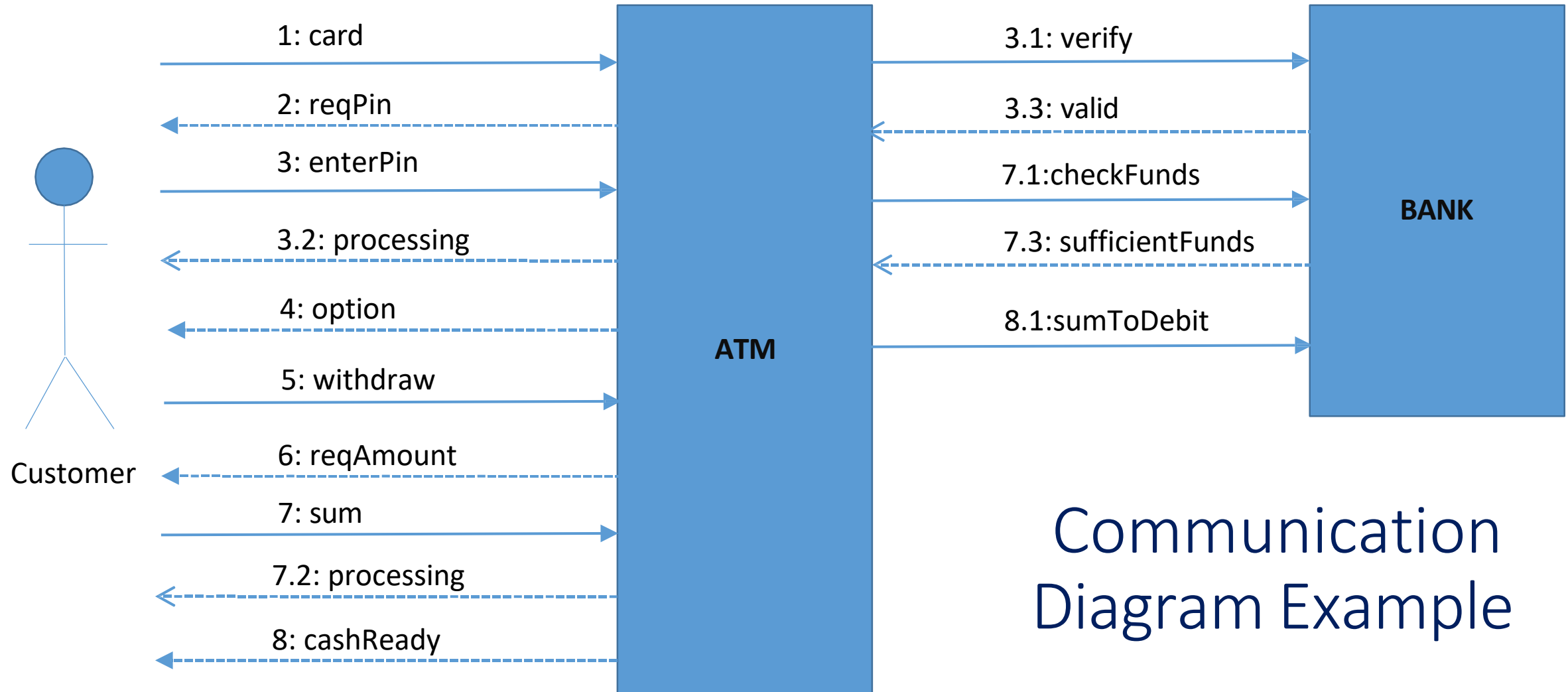
(10 minutes)





ATM Machine





Summary

- By the end of this lecture, you should be able to translate conceptual systems into the different kinds of UML diagram we have looked at
- You should have gained an awareness of which diagrams are appropriate for modelling the different aspects or views of the system
- UML Behaviour Diagrams were presented: Use Case Diagram, State Machine Diagram, Activity Diagram, Sequence Diagram, and Communication Diagram



Thank You!

